MDPI

Article

Verifying Robot IMU Link Integrity beyond Packet Liveness

Author names to be supplied

Author affiliations and addresses to be supplied.
* Corresponding author and email to be supplied.

Abstract

Continuous controller feedback does not establish acceptance or freshness of incoming robot measurements. We examine this distinction through a retrospective audit of three live inertial measurement unit (IMU) records from two robot hosts and a historical hardware replay on a third controller. Four complete captures contain 121,658 valid inputs and 40,380 valid returns. A receiver candidate retaining only checksum bytes below 128 explained 91.37–98.84% of temporally held-out return payloads, compared with 44.43–48.55% for a latest-input candidate. Circularly shifted admission gates preserved the admitted-input count but explained at most 34.77%. Offsets frozen from the original robot record explained 96.30% of returns in a separate session and 95.97% in the third-controller replay. Complete-record analysis retained end-of-session intervals where both candidates failed, bounding the reconstructed mechanism. A historical marker diagnostic and an exhaustive offline comparison of 6,144 valid packets supported the signed-byte rejection boundary. These results distinguish packet validity, publication liveness, and evidence of accepted samples. Individual input acceptance, true sensor age, corrected firmware behavior, and motion consequences remain unmeasured.

Keywords: robot verification; inertial measurement unit; sensor integrity; runtime monitoring; data freshness; checksum; reproducibility

1. Introduction

Robot controllers depend on measurements passing through several software and communication boundaries before they can influence control. A sensor may acquire data correctly, a host may serialize and transmit valid packets, and a controller may publish feedback continuously, while the values available inside the controller fail to follow incoming measurements. These observations concern different parts of the chain. Treating any one of them as evidence for the entire chain can conceal a defect at a boundary that ordinary process or packet monitoring does not observe.

Verification of robotic systems already recognizes the difficulty of connecting component evidence to claims about an integrated system. Surveys distinguish the properties established by testing, simulation, and formal verification, and emphasize the importance of explicit assumptions about the system and its environment [1,2]. In *Robotics*, Fisher et al. discuss these challenges across inspection systems and the use of heterogeneous evidence in assurance arguments [3]. Empirical software studies provide a complementary perspective: the ROBUST dataset documents faults, failures, fixes, and reproducible historical environments across seven Robot Operating System (ROS) software systems [4]. Together, these studies motivate close examination of the observations used to declare a deployed component operational.

This study addresses an observation problem at a robot sensor–controller boundary: what can be concluded when feedback packets remain valid and continuous, but their relationship to accepted source samples is not explicit? The distinction follows the established end-to-end argument that a lower-level mechanism cannot, by itself, establish every application-level property [5]. For sensor links, successful packet generation and return publication are useful checks, but they leave open whether a particular measurement passed the receiver's validation and became current state.

The available evidence is unusually suitable for examining this distinction. An archived engineering investigation contains three live Intel RealSense D455 records from two hosts, a real-data replay on a third controller, and a separate synthetic checksum diagnostic. Bidirectional captures, source records, and version and state context accompany these observations. The diagnostic had already exposed a signed-byte comparison problem during the original investigation. The present study therefore does not claim prospective discovery of an unknown fault. Its research contribution is a reproducible retrospective test of which monitoring observations would miss that fault, which observations discriminate the candidate mechanism, and which conclusions remain unidentifiable even with bidirectional captures.

We ask three questions. RQ1: Do packet validity, continuous return publication, and retrospective value matching establish measurement acceptance or freshness? RQ2: Does a checksum-dependent receiver model explain the archived real-data traces better than an otherwise identical latest-input model, and is that interpretation consistent with the controlled diagnostic? RQ3: Which integrity claims can be verified offline, and which require additional controller instrumentation or physical experiments? We answer these questions with an explicit evidence hierarchy, a temporal holdout comparison, boundary-specific marker analysis, and finite-domain protocol checks. The measured boundary is the IMU input and feedback path of three controllers sharing one firmware family. Actuator execution and closed-loop performance lie outside that boundary.

2. Related Work

2.1. Robot Verification and Runtime Observation

Formal verification and testing establish properties relative to a specification and a model of the relevant environment [1,2]. The challenge in a deployed robot is often deciding whether the specification's observable events correspond to the behavior that matters physically. Fisher et al. distinguish component and system reliability, while Zaki et al. use semantic relationships between component states to support online reasoning about reliability and safety [3,6]. Our study examines a smaller boundary: the inferential gap between a published packet and an accepted measurement. It provides a concrete counterexample for that boundary rather than a new system certification framework.

ROSMonitoring supplies a runtime verification framework that separates monitoring infrastructure from the specification formalism [7]. Its extension, ROSMonitoring 2.0, adds services and ordered topics [8]. Caldas et al. provide field-testing and runtime-verification guidelines, including explicit instrumentation and test-oracle requirements [9]. Our byte-level comparisons instantiate such an oracle at one sensor boundary. Such tools offer an appropriate place to express the distinctions examined here. A monitor of publication frequency can establish liveness, whereas a monitor of sample acceptance needs an event or field that identifies accepted input. Our analysis does not benchmark these frameworks or suggest that they inherently conflate the two; it studies the evidence available to a monitor when the protocol itself omits that identification.

2.2. Freshness and Communication Integrity

Age of information measures the time elapsed since generation of the status information available to a recipient [10,11]. Its definition depends on identifying the generating update. In the present protocol, the returned counter is generated by the return publisher, and the original sample timestamp is absent. Consequently, a numerical match to an earlier acceleration value cannot identify the generating sample when values repeat. We use a more limited, explicitly observable recent-match age and do not label it sensor age or end-to-end latency.

ROS 2 quality-of-service specifications separately describe deadline, liveliness, and lifespan [12]. This separation is relevant conceptually, although the studied path uses a custom User Datagram Protocol (UDP) payload rather than ROS 2 transport. A timely publisher can remain alive while reporting cached state. Likewise, payload checksums address specified byte-error patterns rather than freshness or correct interpretation. Maxino and Koopman evaluate checksum effectiveness for embedded control networks and distinguish additive sums from stronger error-detection codes [13]. The studied protocol uses an eight-bit additive sum, despite the historical files calling it "CRC". We use *checksum* throughout because no cyclic polynomial calculation is present.

The contribution relative to this literature is an auditable case analysis joining packet semantics, a controlled receiver diagnostic, and the limits of passive freshness inference. It includes a temporal comparison of simple receiver candidates and a complete checksum-byte sweep of an offline model. These are complementary to broad robot fault datasets and general runtime monitoring frameworks; they do not establish a new general-purpose detector or a prevalence estimate for this fault class.

3. Materials and Methods

3.1. Study Design and Evidence Provenance

The core receiver analysis reuses engineering records acquired on 12 and 16 October 2026; auxiliary ROS recordings are evaluated separately. A read-only resource audit on 19 October attempted SSH access to all 63 device entries in two online-roster snapshots. Forty-seven hosts permitted traversal of declared data roots and bounded content inspection; 16 were inaccessible. Directory and permission exclusions were recorded. Relevant existing files were retrieved for offline analysis. No sensor acquisition, controller command, service action, configuration change, or replay transmission was performed for this study. The analysis was retrospective and was not preregistered. Cases were selected by availability of bidirectional captures and source/version context, rather than by random sampling.

R1 and R2 denote the original live records from two hosts. R1-repeat is an earlier, separately acquired session on the same physical robot as R1. H1 is a historical replay into a third controller, using D455 measurements previously recorded on the R2 host. H1 does not establish a live sensor link on that third host. A source-only loopback record from 12 October supplies H1's data lineage, not an additional controller replication.

We retained the original approximately 60 s R1/R2 measurement windows as primary analyses. The expanded analysis used every return publication in all four complete captures, with full input prehistory and no endpoint optimization. D1 remains a separate historical synthetic diagnostic on R1; O1 comprises new offline comparisons. A separate ROS-record audit evaluates recording semantics and does not add controller replications. Table 1 separates these evidence units and their dependence.

Table 1. Evidence units and the conclusions they can support. Packet counts are technical observations within a case, not independent robot replications.

Unit	Origin	Available evidence	Principal boundary
R1	Live D455, robot 1	12,615 full-capture inputs / 8,575 returns; original 60 s window retained	Input near 101 Hz; one acquisition session
R2	Live D455, robot 2	72,565 inputs / 10,562 returns; original 60 s window retained	Input near 402 Hz; different acquisition path
R1-repeat	Live D455, robot 1	Earlier session: 18,388 inputs / 10,684 returns; source CSV	Repeated session, not an independent robot
H1	Recorded-data replay, robot 3	18,090 inputs / 10,559 returns; replay log and source recording	Independent controller; reused R2-host sensory source
L1	Host-only loopback, R2 host	12,040 source and receiver packets; fresh archive retrieval	Sender/decoder control; source of H1, not another controller
D1	Historical diagnostic, R1	Five checksum classes, three marker windows each; capture and generator source	Synthetic input; one controller; fixed marker order
ROS audit	Existing recordings	10 unique MCAP files; 32 IMU streams / 17,645 messages	Recording semantics; no additional defect replication
O1	New offline computation	6,144 valid packets; receiver comparison and corruption controls	Model verification only; no controller repair test

The controller firmware was recorded as version 100343032 in all three controller contexts. The archived receive-function disassembly is associated with binary SHA-256 beginning `9613ebd90918` (full digest in Supplementary Information S1). This is a version-specific observation, not a statement about other releases. R1 used a Linux Industrial I/O acquisition path; R2 used a librealsense sender configured for 400 Hz acceleration. The R2 sender recorded D455 firmware 5.17.3.10. Exact historical dependency versions were not completely preserved for both paths. Source files and available version records were hashed as provenance evidence; missing versions were not inferred from current installations.

Both paths encoded acceleration as $(a_x, a_y, a_z) = (a_{z,\mathrm{opt}}, -a_{x,\mathrm{opt}}, -a_{y,\mathrm{opt}})$, consistent with the forward–left–up axis convention in REP-103 [14]. The archive does not supply a multi-orientation calibration experiment, so the transformation is a documented implementation convention rather than a physically validated axis calibration. Acceleration is expressed in $\mathrm{m\,s^{-2}}$. No inference concerning orientation accuracy or gyro integration is made. The source CSV files contain 44,560 and 88,645 rows, respectively, spanning more than the selected packet windows. R2 has one initial row with unavailable gyro values; its acceleration is finite and gyro values are absent from the tested payload.

The historical measurement and diagnostic records report disabled arms, no motion commands, and zero dynamic-floating configuration flags. Subsequent records report restoration of the original configurations and no IMU teleoperation acceptance. These statements characterize the archived procedure; they are not a fresh physical inspection. The archived generator contains network transmission and the surrounding maintenance scripts contain state-changing operations. They were inspected as source evidence and were not executed during the present analysis. R1-repeat contains a recorded handoff-validation exception; its configuration restoration was subsequently recorded. We retain that exception and make no successful handoff or teleoperation claim.

3.2. Packet Contract and Observable Properties

Each application payload has 31 bytes: the two-byte ASCII header `FX`, a little-endian 32-bit integer counter at offsets 2–5, three little-endian IEEE 754 binary64 acceleration values at offsets 6–29, and a checksum byte at offset 30. The contract is

$$c = \left(\sum_{k=2}^{29} b_k \right) \bmod 256. \tag{1}$$

The studied senders used counters in the range 1–1,000,000. The payload contains neither the source timestamp nor an explicit controller acceptance flag. Input and return counters are independent; equality of their numerical values is not an acknowledgment relationship.

We separate four properties. *Packet validity* means that a captured payload satisfies the length, header, counter range, finite-value, and checksum checks. *Publication liveness* means that return packets continue to arrive with measured intervals and counter continuity. *Acceptance* means that the receiver admitted a particular input to its state update. *Freshness* concerns the generating time or identity of the sample actually used. Only the first two are directly observed by a validity-and-frequency monitor. Figure 1 shows this boundary. The analysis verifies application payloads and does not constitute a complete Ethernet, IP, or UDP integrity audit.

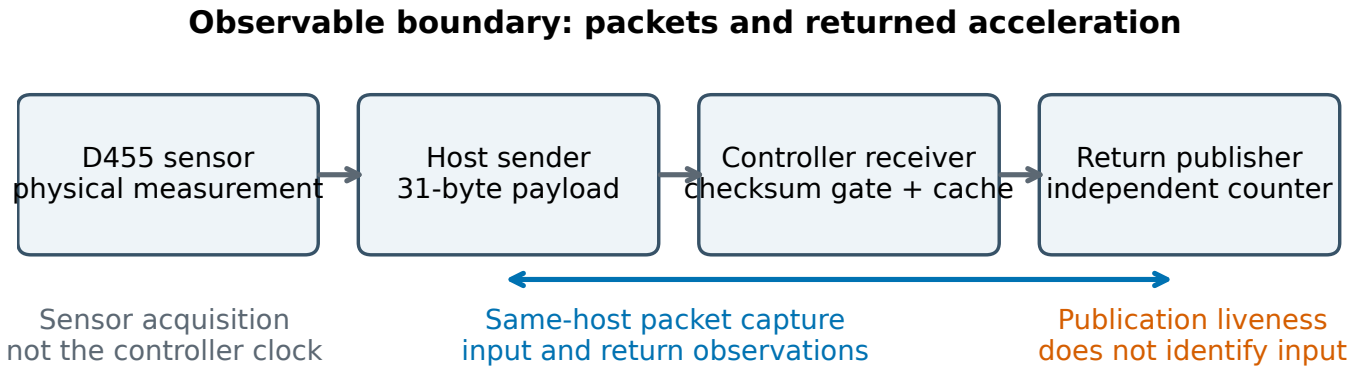


Figure 1. Observation boundary for the robot inertial measurement unit (IMU) link. Bidirectional packet capture observes serialized inputs and published returns. The controller's validation gate and current-sample identity are not exposed directly. A return counter identifies publications rather than accepted source samples.

3.3. Independent Capture Parsing and Value Matching

An independent Python reader parsed the original classic Ethernet/IPv4/UDP captures. Timestamps were retained as integer nanoseconds, avoiding loss of precision from converting epoch-scale values to floating point. The parser rejected truncated frames and fragmented IP records, checked each selected payload, and verified every decoded measurement-table row against its original captured timestamp and bytes. It then sorted observations by timestamp for causal matching. R1 contained two cross-direction timestamp reversals in capture order, with maximum magnitude 1 μs; R2 contained nine, with maximum magnitude 13 μs. The sorting and later timing sensitivity checks address ordering ambiguity without changing the raw records.

Let input i be observed at host capture time t_i with acceleration bytes x_i, and return j at r_j with bytes y_j. We defined the recent-match age as

$$A_j = r_j - \max\{t_i : t_i \le r_j,\ x_i = y_j\}. \tag{2}$$

The comparison used the exact 24 acceleration bytes, excluding header, counters, and checksum. If no earlier match existed, the return was marked unmatched. Full-capture prehistory was used, including for the first selected return. We repeated the calculation using only earlier inputs whose checksum was below 128. We also linked selected inputs to source CSV rows by source counter and binary64 acceleration, and counted compatible earlier inputs in the preceding 1 s to quantify sample-identity ambiguity.

A_j measures time since the most recent compatible input observed at the host. When payloads repeat, several inputs may be compatible and the actual generating sample may be older. Even with complete capture, this statistic is not a measurement of source-to-controller or source-to-control latency. It also excludes the acquisition-to-host interval. We therefore report it as an observable descriptor without asserting a control deadline. Adjacent returns with identical acceleration bytes were counted separately; identical values alone were not labeled stale because a stationary sensor can legitimately repeat a value.

Packet rates were computed as $(N-1)/(t_N - t_1)$. We report interval quantiles, maximum intervals, counter gaps, checksum classes, exact matches, and identical-payload runs. One-second blocks wholly contained within each return window describe temporal heterogeneity. No packets were removed as statistical outliers, and the two cases were analyzed separately.

3.4. Receiver Candidates and Temporal Holdout

We compared two receiver candidates that share the same publication times and differ only in admitted inputs. Model M_{any} retains the most recent captured input. Model M_{low} retains the most recent input with checksum $c < 128$, corresponding to the boundary implied by a signed-byte comparison. At return time r_j, the predicted payload is that retained at $r_j - \delta$, where δ is an unknown readout offset. This offset absorbs unmodeled scheduling and observation differences; it is not an estimate of processing latency.

For each case and each candidate, δ was selected from 0 to 20 ms in 0.25 ms steps to maximize exact agreement in the first 20 s. Ties favored the smaller offset. The selected offset was fixed before computing agreement in the remaining approximately 40 s. The primary comparison is the difference in held-out exact-payload agreement. Candidate selection was informed by the known historical diagnostic, so the holdout tests temporal consistency within a trace rather than constituting an independent, blinded discovery test.

Sensitivity analyses changed the training duration to 10 or 30 s and shifted all return timestamps by -100, -20, 0, 20, or 100 μs before repeating offset selection. The offsets exceed the observed capture-order reversals. We also evaluated M_{high}, which retains only inputs with $c \ge 128$, as a contrasting checksum-specific candidate. Whole-record agreement curves over the offset grid are descriptive and distinct from the primary held-out results. No candidate includes packet-specific fitted offsets.

3.5. Complete-Capture Replication and Admission Controls

The expanded analysis included all captured returns, spanning 93.01–116.19 s per record. Every captured input was linked to source counter and acceleration bytes. H1 additionally required equality with the source packet indexed modulo the 12,040-packet recorded sequence, allowing its historical counter rewrite. A fresh copy retrieved from the R2 host matched H1's source recording byte for byte. Repeated playback does not create independent sensor observations.

For each complete record, we repeated the first-20-s training and remaining-time holdout over the same offset grid. Two periodic controls retained even or odd source counters, respectively. Thirty-two circular shifts of the low-checksum admission mask preserved its exact admitted count and circular run structure. Shift offsets were 32 evenly spaced integer positions from 1 to $N-1$; the unshifted mask was excluded. Each control selected its own offset on training data. The minimum–maximum held-out range describes these controls; it is neither a confidence interval nor a randomization test.

To avoid fitting the added records, a second analysis fixed the any-input and low-checksum offsets at 1.0 and 0.5 ms. These values came from the original R1 measurement-window training. We evaluated all returns in R1-repeat and H1 without selecting offsets on either record. We also evaluated their archived measurement windows, identified by unique return counters and verified captured bytes. These retrospective checks test transfer of a known mechanism; they are not blinded fault discovery.

All complete one-second return blocks were retained to expose temporal model failures. Long captured-input gaps outside the return intervals were catalogued separately. Missing source counter ranges across those gaps were not classified as network loss. Historical restoration/reboot timelines were inspected to contextualize late residuals, without assigning an unobserved internal cause.

3.6. Historical Boundary Diagnostic

D1 used a generator targeting checksum bytes 0x00, 0x7F, 0x80, 0xC8, and 0xFF. Each class was transmitted in three nominal 80 ms windows, separated by low-checksum keepalive traffic. Each window contained 32 inputs at nominal 400 Hz. The nominal acceleration markers were $a_x = 0.25$, 0.375, 0.5, 0.625, and 0.75 $\mathrm{m\,s^{-2}}$, respectively, with $a_y = 0.03125$ and $a_z = 9.80665\ \mathrm{m\,s^{-2}}$. A least-significant mantissa byte of a_x was adjusted to realize the target checksum without materially changing the marker value.

We verified the diagnostic CSV against the captured input payloads and recomputed every checksum. Marker publications were identified using $|a_x - a_{x,\mathrm{marker}}| < 10^{-10}$, $|a_y - 0.03125| < 10^{-12}$, and $|a_z - 9.80665| < 10^{-12}$. These tolerances distinguish the marker levels while allowing the prescribed mantissa adjustment. For each window, the analysis counted return publications and publications carrying its marker from the first captured input until 2.5 ms after the last input. We also searched the full diagnostic return capture for each marker, so the conclusion did not depend solely on a short window ending before a delayed return.

This diagnostic has a positive observation condition (low-checksum markers), negative conditions (high-checksum markers), repeated technical windows, and continuing return publication. However, checksum class, marker magnitude, and order are coupled in the historical design. The three windows per value are repeated exposures on one controller, not three independent controllers. Their interpretation is strengthened by the disassembly and offline comparison but is not a randomized checksum-only experiment.

3.7. Finite-Domain Verification and Integrity Controls

The archived receive-function disassembly shows an unsigned accumulated byte sum compared with a sign-extended received checksum byte. For valid checksum c, the reconstructed comparison is

$$U(c) = S(c), \qquad U(c) = c, \quad S(c) = \begin{cases} c, & 0 \le c < 128, \\ c - 256, & 128 \le c \le 255. \end{cases} \tag{3}$$

Equation 3 accepts precisely the lower half of the valid checksum domain. A corrected offline comparison treats both operands as unsigned bytes. The reconstruction concerns the observed comparison instructions; it does not establish all behavior of the proprietary controller binary.

We generated all 256 valid checksum values for six counter bases (1, 127, 128, 255, 256, and 1,000,000) and four acceleration bases (0.125, -1.25, 0.5, and 9.80665 $\mathrm{m\,s^{-2}}$), giving 6,144 packets. Adjusting the least-significant mantissa byte preserved finite acceleration; every packet was decoded and revalidated. An independent minimal C++ comparison was compiled with both `-fsigned-char` and `-funsigned-char`, checking plain-character behavior and explicit signed and unsigned interpretations. No generated packets were sent to hardware.

Two corruption controls separated correct checksum implementation from the error-detection strength of the contract. First, each of the 224 payload bits covered by Equation 1 was flipped individually without updating the checksum. Second, 255 nonzero modular perturbations added a byte delta at one payload position and subtracted the same delta at another. These controls test specified byte patterns; they do not estimate environmental bit-error rates or cryptographic security.

3.8. Auxiliary Audit of Existing ROS IMU Records

Among successfully inspected files after the resource traversal, we selected the smallest finalized, size/mtime-stable MCAP file of at most 64 MiB per accessible host containing at least 100 messages on a standard `sensor_msgs/msg/Imu` channel. The rule uses file size and schema/count metadata, not timing or measurement outcomes. It was applied after exploratory inventory rather than preregistered; preliminary plans were retained and superseded by the complete-inventory selection. We downloaded whole files, verified stable source size/mtime and local SHA-256 hashes, deduplicated by hash, and decoded every qualifying IMU channel with available chunk-CRC checks. The resulting sample is deliberately small-file biased, and host storage does not establish physical acquisition identity.

The MCAP specification distinguishes recording and publication times and allows an unused sequence field to be zero [15]. The ROS IMU schema provides a header timestamp and measurements, without a controller accepted-input identifier [16]. In file order within each channel, we counted timestamp reversals, repeated positive header stamps, zero sequence fields, and identical adjacent six-component acceleration/angular-velocity values. For comparison, an intentionally naive rule flags every adjacent sequence pair whose modulo-2^{32} increment is not one. Without a populated counter and its producer contract, those flags cannot identify lost messages. Timestamp domains were kept separate; no source-to-controller delay was estimated. Supplementary Information S1 specifies selection, decoding, and per-record results; S3 provides shifted derivatives and offline code.

3.9. Statistical and Reproducibility Policy

The receiver-case evidence comprises three controllers, three live-acquisition sessions on two hosts, one historical replay, and a controlled diagnostic on R1. The R1 sessions share hardware, and H1 reuses an earlier sensory source from the R2 host. Thousands of packets are temporally dependent observations, and the source-rate difference is confounded with robot and acquisition path. We therefore report case-specific effect sizes, empirical quantiles, full counts, within-record block ranges, and sensitivity ranges. We do not compute packet-level significance tests, population confidence intervals, or a pooled robot reliability estimate. Finite-domain counts are exhaustive over the specified offline

domain, so sampling intervals would be inappropriate. No multiplicity-adjusted significance claim is made because no hypothesis-testing family is used.

All distributed analysis scripts are offline and deterministic, with no random sampling. The software environment, file hashes, parameters, marker windows, prediction curves, and numerical results are saved in the reproducibility package. Analyses used Python and NumPy; figures used Matplotlib and pandas. Exact executed versions are recorded with the package. The public-oriented derivative replaces network addresses and absolute capture dates while preserving payload bytes, ordering ambiguities, and relative timing. Original captures and proprietary context remain in the local restricted evidence set pending release authorization.

4. Results

4.1. Continuous Valid Feedback Did Not Identify Accepted Inputs

All 30,162 selected input packets and 11,061 selected return packets passed the application-payload checks. Both input counters and return counters had no gaps within their respective windows. The input rates differed by almost a factor of four, while return publication remained near 92 Hz (Table 2). A frequency or counter-continuity check would therefore have observed an active return publisher in both records, without establishing how many input samples had been accepted.

Table 2. Real-data observations in the archived measurement windows. Recent-match age follows Equation 2; it is not sensor age or latency. No inferential uncertainty intervals are attached to these within-record descriptors.

Observation	R1	R2
Input / return packet count	6,053 / 5,522	24,109 / 5,539
Input / return rate (Hz)	100.87 / 92.01	401.85 / 92.31
Maximum input / return interval (ms)	10.382 / 11.103	3.581 / 11.025
Inputs with checksum ≥ 128, count (%)	3,077 (50.83)	12,147 (50.38)
Distinct input acceleration payloads	194	1,143
Returns with an earlier exact payload match (%)	100	100
Recent-match age median / 95th percentile (ms)	10.910 / 52.375	3.360 / 11.514
Recent-match age maximum (ms)	168.112	32.466
Identical adjacent return payload pairs (%)	51.37	5.69
Longest identical return run (publications)	16	3

Every selected input matched its source CSV row exactly, and every selected return matched an earlier captured acceleration payload, including when candidate matches were restricted to low-checksum inputs. This reproduces the reassuring observation underlying the historical packet-roundtrip assessment, but not a full acceptance conclusion. In R1, 4,191 of 5,522 returns (75.90%) had more than one matching input in the preceding 1 s; the corresponding count in R2 was 3,434 of 5,539 (62.00%). The independent counters and repeated acceleration values prevent assignment of a unique accepted source sample to these returns. The observed high-checksum fraction is therefore reported as a property of the input stream, not a measured input-drop percentage.

Figure 2 shows that the two records also differed in repeat behavior and recent-match age. The R1 record contained an identical-payload run of 16 return publications despite continuous counters. Such a run is compatible with retaining an earlier state, but stationary measurements and quantization also produce repeated payloads. Recent-match age is therefore useful for describing the observable relationship between streams, while its value alone does not establish controller freshness.

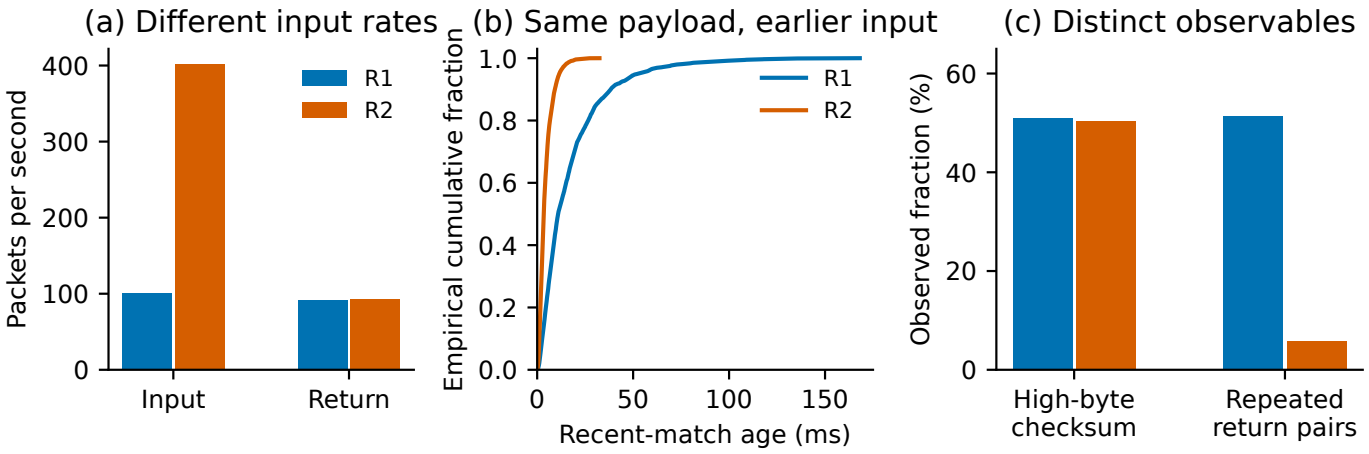


Figure 2. Different observables from two real sensor records. (a) Input and return packet rates. (b) Empirical cumulative distributions of recent-match age, using full-capture prehistory. (c) Input high-checksum fractions and identical adjacent return-payload fractions. Bars summarize individual records without error bars; R1 and R2 are not randomized source-rate conditions.

4.2. A Checksum-Dependent Receiver Better Explained Later Payloads

The checksum-filtered candidate showed substantially higher held-out agreement than the latest-input candidate in both cases (Table 3). The absolute differences were 50.61 percentage points in R1 and 48.65 percentage points in R2. Because the candidates used the same observed return times and differed only in the retained input class, these comparisons provide a specific test of the receiver-state explanation. They do not transform model agreement into an acknowledgment count.

Table 3. Primary temporal holdout comparison. Offsets were selected using the first 20 s and fixed for the remaining approximately 40 s. Agreement is exact equality of the 24 acceleration bytes.

Case	Receiver candidate	Selected offset (ms)	Test returns	Test agreement (%)
R1	Latest input, any checksum	1.00	3,681	48.11
R1	Latest input, checksum < 128	0.50	3,681	98.72
R2	Latest input, any checksum	0.75	3,692	45.75
R2	Latest input, checksum < 128	0.75	3,692	94.39

The result persisted across the specified training durations and timestamp shifts. Held-out agreement for the low-checksum candidate ranged from 98.33% to 98.89% in R1 and from 94.39% to 94.87% in R2; the any-checksum candidate ranged from 47.69% to 48.79% and from 44.42% to 45.75%, respectively. The high-checksum-only candidate remained below 0.14% in both cases. Figure 3 displays the full-record offset dependence, making the sensitivity to the assumed observation offset visible rather than presenting a single fitted number as intrinsic timing.

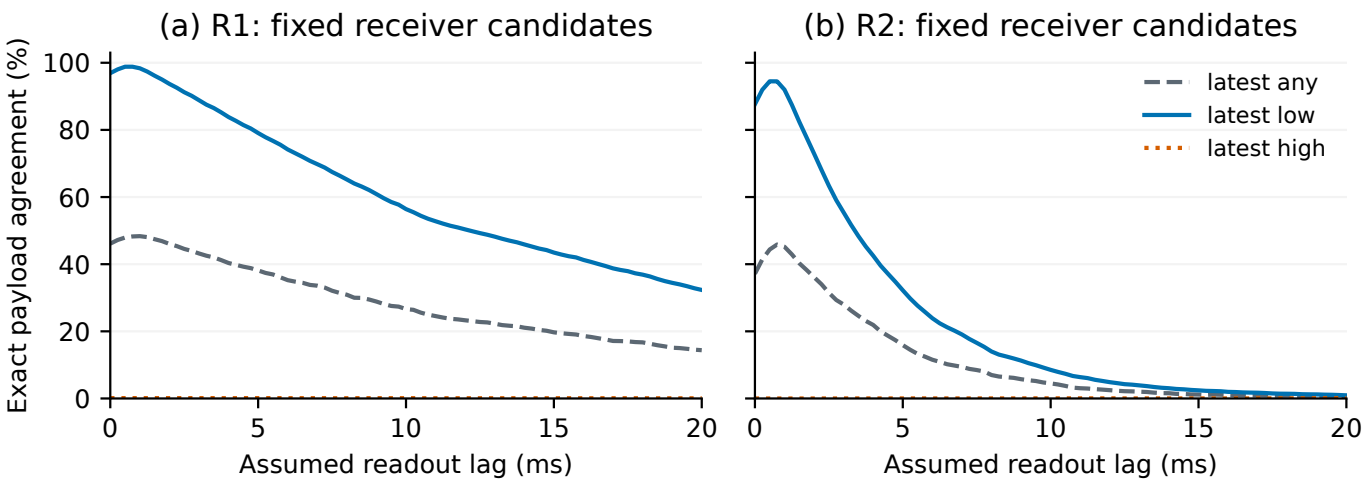


Figure 3. Exact agreement over the fixed offset grid for the original R1 and R2 measurement windows. The low-checksum receiver candidate is compared with any-checksum and high-checksum candidates. These curves describe the complete original measurement windows; their training/held-out estimates are in Table 3. Offsets are nuisance alignment parameters and are not measured processing delays.

The low-checksum candidate was not exact: approximately 1.28% and 5.61% of held-out returns remained unexplained. Internal readout scheduling, capture-to-receive timing, or other unmodeled buffering could contribute, but their individual roles were not observed. The stronger fit supports the checksum-dependent interpretation within the available traces; it does not identify a complete controller execution model.

4.3. Complete Captures Supported Transfer and Retained Transition Failures

Across four complete captures, all 121,658 inputs and 40,380 returns passed the application-payload checks. Every input matched its source record, and every return matched an earlier input payload. Return counters remained consecutive in every record. All long captured-input gaps occurred outside the return intervals; no such gap was excluded from a model evaluation. Table 4 reports complete-capture counts separately from the original measurement-window counts.

The separately analyzed host-loopback control L1 delivered all 12,040 source packets with exact bytes, including 5,953 high-checksum packets. Its packet recording was byte-identical to H1's source. This establishes source and host-decoder consistency for both checksum halves; it does not establish hardware admission of those packets.

Table 4. Expanded complete-capture analysis. Offsets were fitted to the first 20 s of each record for the two rightmost columns. Full input counts include prehistory and traffic after return publication ended.

Record	Full inputs	Full returns	Return span (s)	Any, test (%)	Low, test (%)
R1	12,615	8,575	93.01	48.55	98.84
R2	72,565	10,562	114.35	44.43	91.37
R1-repeat	18,388	10,684	116.19	48.11	95.90
H1 replay	18,090	10,559	114.44	47.74	95.41

The low-checksum candidate exceeded both periodic controls in each complete-record holdout (Figure 4a). Circularly shifted gates, despite preserving the same admitted-input count, reached at most 34.77% agreement. Thus, reduced update frequency alone did not reproduce the observed checksum alignment under these controls. This is a descriptive discrimination among specified receiver candidates, rather than a probabilistic test of every competing mechanism.

With offsets frozen from the original R1 window, agreement on all returns was 96.30% for R1-repeat and 95.97% for H1. The corresponding any-input values were 47.97% and 47.87% (Figure 4b). In their archived measurement windows, the low-checksum agreement was 98.88% over 5,512 returns and 98.63% over 5,530 returns. H1 extends the consistency evidence to a third controller, while its recorded source remains distinct from a third live acquisition.

The complete captures also exposed a material qualification. R2's low-checksum holdout agreement fell from 94.39% in the original window to 91.37% in the complete-capture evaluation. Late one-second blocks in R2, R1-repeat, and H1 reached zero agreement for the fixed low-checksum candidate (Figure 4c). These blocks occurred near logged restoration reboot requests, after the archived measurement windows. They were retained in all complete-record results. The coincidence provides procedural context but does not identify the internal transition responsible for the model failures. The checksum candidate therefore describes the measurement regime better than it reconstructs all end-of-session behavior.

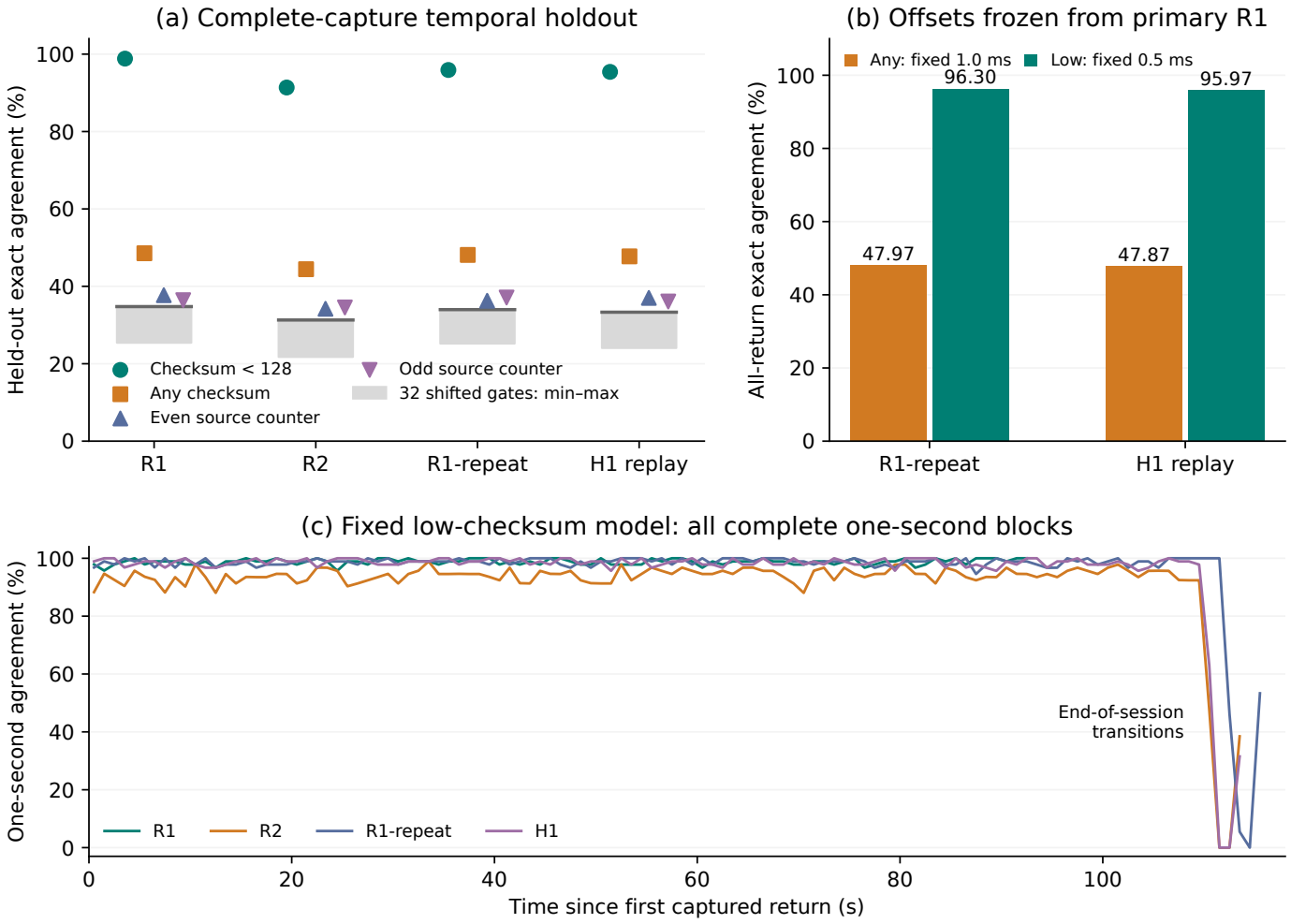


Figure 4. Expanded records and stronger controls. (a) Complete-capture temporal holdouts; gray bands are the minimum–maximum across 32 circularly shifted gates, not inferential intervals. Even/odd controls retain alternate source counters. (b) Evaluation of all returns in the added records using offsets fixed on the original R1 window. (c) Fixed low-checksum agreement for every complete one-second block, including late failures near historical restoration requests. Time starts at each record's first captured return.

4.4. Boundary Markers Distinguished Publication from Admission

All 3,025 logged diagnostic inputs were present in the D1 capture and had valid checksums. The full capture contained 3,914 input packets, including real-source traffic

around the synthetic interval, and 1,414 return packets. For each tested checksum, 96 marker inputs were present in three windows (Table 5). Both low-checksum markers appeared in all their windows, producing 22 publications per marker. No high-checksum marker appeared anywhere in the diagnostic return capture.

Table 5. Historical diagnostic D1, on R1 only. Marker publication counts are not accepted-input counts. Each checksum class had three repeated technical windows on the same controller.

Input checksum	Input markers	Windows	Windows with marker	Marker publications
0x00	96	3	3	22
0x7F	96	3	3	22
0x80	96	3	0	0
0xC8	96	3	0	0
0xFF	96	3	0	0

Each of the nine high-checksum windows nevertheless contained seven or eight return publications, totaling 66 publications during those windows. Thus, publication liveness coexisted with the absence of the currently transmitted marker. Figure 5 visualizes the repeated marker sequence and its selective appearance. This observation directly answers RQ1 for the tested diagnostic: a continuing return stream did not establish propagation of the current input marker.

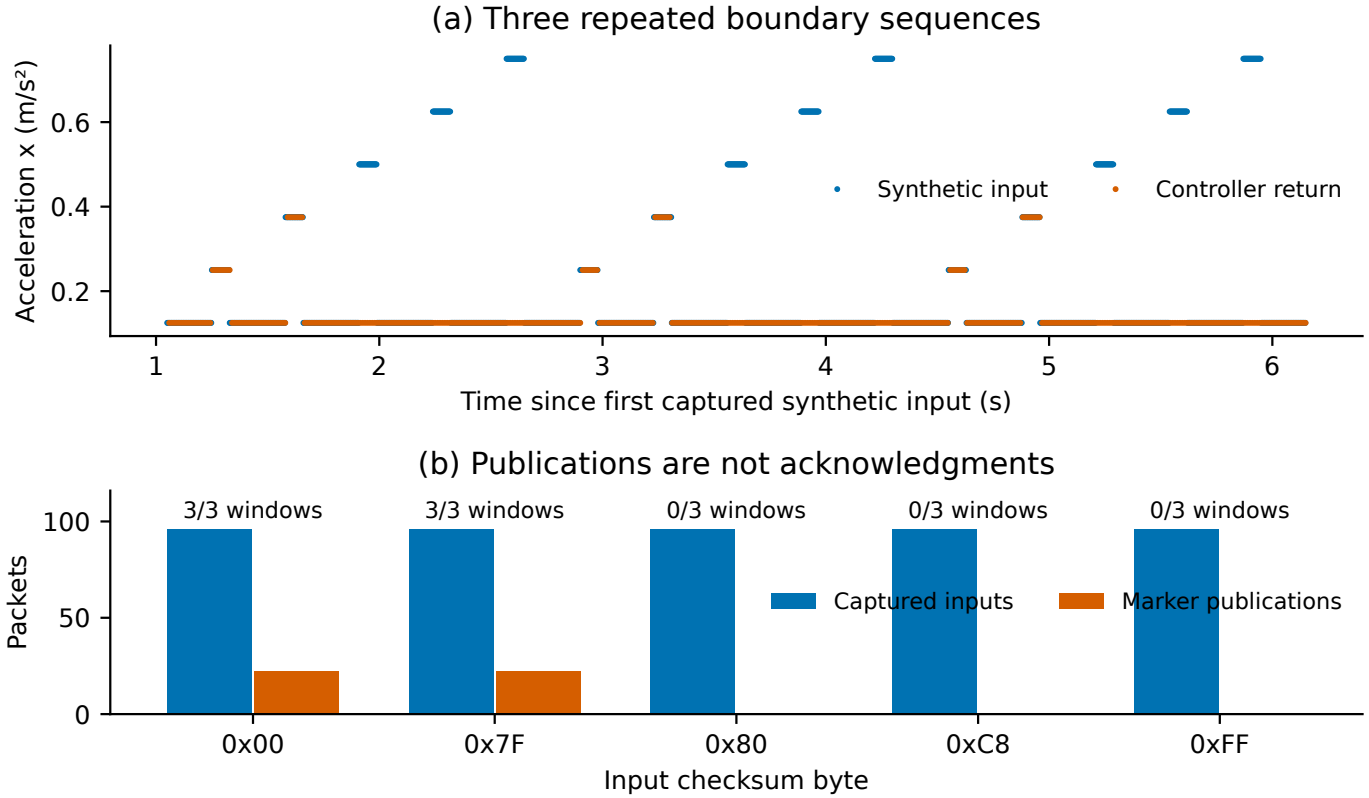


Figure 5. Historical synthetic diagnostic, analyzed separately from real sensor data. (a) Captured marker inputs and returned acceleration over three boundary sequences. The approximately 0.125 m s^{-2} level is keepalive traffic. (b) Input counts and marker-publication counts for each checksum value, with the number of windows displaying the marker. The apparent reduction from 96 inputs to 22 low-checksum publications includes rate conversion and is not an input-loss estimate.

4.5. Offline Checks Reproduced the Boundary and Exposed a Separate Limitation

The reconstructed signed-byte comparison accepted 3,072 of 6,144 valid offline packets, precisely those with checksum bytes below 128. The unsigned comparison accepted

all 6,144. The largest acceleration perturbation needed to realize the complete checksum domain was 4.44×10^{-13} m s^{-2}. The independent C++ check accepted 128 of 256 byte values through plain `char` when compiled with `-fsigned-char`, and all 256 with `-funsigned-char`; explicit unsigned conversion accepted all 256 under both settings. These experiments reproduced an implementation-level boundary without imposing a meaningful acceleration change on the offline values.

The checksum detected all 224 individual covered-bit flips but failed to detect all 255 balanced two-byte perturbations (Figure 6). This is a distinct limitation of the additive contract. Fixing signedness in the comparison would correct the modeled rejection of valid high-byte checksums; it would not convert the checksum into a stronger corruption detector. No corrected firmware was installed or tested as part of this study.

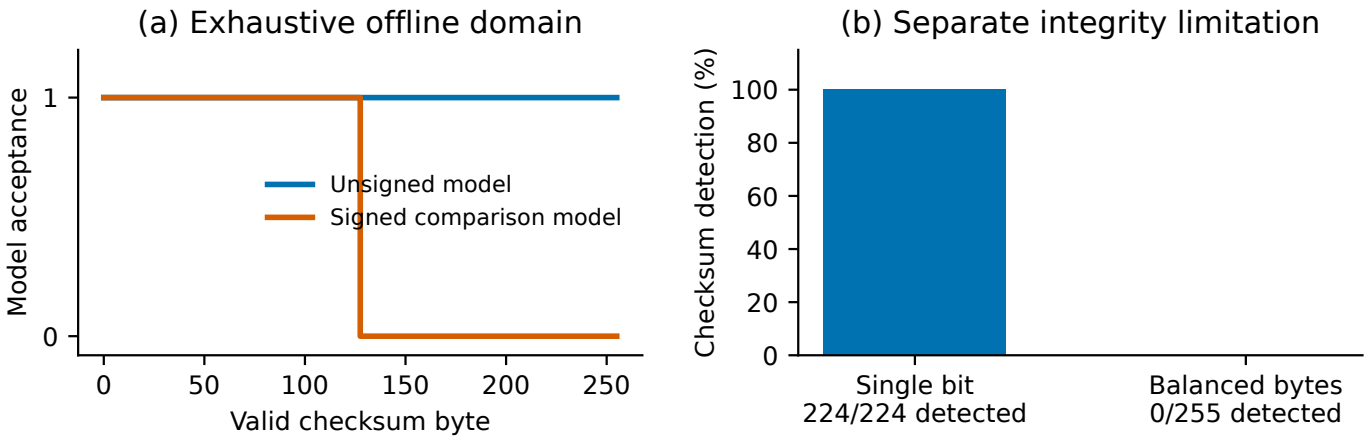


Figure 6. Offline protocol controls, with no hardware transmission. (a) Acceptance across all 256 checksum values; each value was instantiated in 24 counter/acceleration combinations. (b) Detection of the explicitly enumerated corruption patterns. Exhaustive counts describe the specified domain and do not estimate real-world corruption probabilities.

4.6. Recorded ROS Fields Did Not Establish Message Continuity

The auxiliary audit included 17,645 IMU messages across 32 streams in 10 unique MCAP files retrieved from distinct hosts. All 17,645 sequence fields were zero. Applying the naive unit-increment rule would flag 17,613 adjacent pairs, although zero is permitted by the format; these are unsupported loss indications, not observed packet losses. The header checks found 0 repeated and 0 reversed positive stamps. 722 adjacent pairs had identical measurement values despite advancing header stamps. Neither repeated values nor advancing stamps alone identify the generating physical sample. This auxiliary result concerns recording semantics; it does not test the proprietary receiver's admission behavior.

5. Discussion

5.1. What the Combined Evidence Establishes

The central result is an observed separation between a live publication channel and propagation of current sensor inputs. Real-data comparisons, third-controller replay, historical marker behavior, and the reconstructed byte comparison support the same checksum-dependent explanation during the measurement regime. Their evidential roles differ: the captures show what was published, the diagnostic shows that specific current markers failed to propagate while publications continued, and the offline analysis verifies the consequences of the observed comparison semantics. Combining these roles is more informative than treating a successful numerical roundtrip as a single pass/fail result.

The interpretation also has credible alternatives and residual uncertainty. A latest-value publisher naturally emits fewer returns than inputs when acquisition is faster, so the input/output rate ratio alone is not evidence of rejection. Repeated values can arise from a stationary sensor and finite resolution. In D1, marker magnitude and checksum class were not independently randomized, so marker behavior alone would not exclude value-dependent processing. The periodic and shifted-gate controls further weaken an explanation based only on fewer state updates. The signed comparison supplies a mechanism tied to the checksum boundary. Its stronger fit across sessions favors that mechanism, while the late transition failures bound its explanatory scope. These converging observations favor the receiver defect while leaving the unexplained model residuals and precise internal scheduling unresolved.

This case instantiates the distinction between component evidence and integrated assurance emphasized in prior robotic verification work [3,6]. It also illustrates an operational consequence of the end-to-end argument [5]: verifying a sender and a return publisher does not establish the missing receiver-state property. The general principle is established; the contribution here is the traceable measurement procedure and empirical counterexample at a real robot IMU boundary. The result should not be interpreted as a prevalence finding for commercial controllers, ROS, or D455 devices.

5.2. Implications for Sensor-Link Validation

A useful validation record should report at least packet validity, publication liveness, receiver admission evidence, and sample identity/freshness separately. Table 6 gives an example of the claims supported at the studied boundary. A pass at one layer should not silently fill an unobserved layer. In a controller that exposes only cached values and a new publication counter, passive monitoring has an identification limit even if every returned payload can be matched to history.

Table 6. A claim-oriented interpretation of the available checks. The final two rows concern measurements absent from this study.

Check or observation	Supported claim	Unsupported extension
31-byte payload and checksum validation	Captured application packets satisfy the contract	Controller accepted every input
Continuous returns and consecutive return counters	Return publication remained active	Values came from the newest source sample
Earlier exact acceleration match	A compatible prior input was observed	Unique source identity or true age of information
Boundary marker plus continuing publications	Specific current markers did not propagate in D1	Fleet-wide rejection probability
All-zero optional MCAP sequence field	No usable continuity counter is recorded	Adjacent zero pairs prove message loss
Offline unsigned comparison	Modeled valid-byte rejection is removed	Corrected hardware is validated
Accepted-source identity and source timestamp	Would enable more direct freshness inference	Not available in the present protocol
Physical controller/actuator response	Would test downstream control consequences	Not measured by packet analysis

An explicit last-accepted input identifier and source timestamp would make freshness monitoring more informative. A design could additionally expose rejection counters classified by length, checksum, sequence, and semantic checks. These changes are recommendations derived from the observation limit; their effectiveness was not measured here. Existing runtime monitoring infrastructure could evaluate such fields, and ROS 2's distinction between deadline and liveliness offers a useful vocabulary for specifying the required properties [7,8,12]. Neither a middleware setting nor a stronger checksum alone can create an absent relationship between published state and accepted source identity.

For embodied or learning-based robot systems, the same distinction may matter when observations are logged for policy training or consumed during execution. A pipeline can have apparently normal topic rates while some measurements repeat because of rejection or caching. This study does not evaluate any learned policy, training dataset, or task-success outcome. Its relevance is as a measurement prerequisite: downstream experiments should establish what the recorded or consumed observation represents before attributing a task-level failure to a model.

5.3. Limitations and Discriminating Follow-up Experiments

The auxiliary ROS sample was selected by small file size and accessible storage, with incomplete physical acquisition context and no clock calibration. It supports a field-semantics audit, not a fleet prevalence estimate or an enlarged receiver-fault sample. External validity of the receiver case is limited by three controllers sharing one firmware family, short stationary records, and a controlled diagnostic on only R1. The additional R1 session repeats hardware, and H1 reuses R2-host measurements rather than adding a live sensor host. The different input rates cannot be interpreted as a causal rate intervention because host acquisition and robot identity also differ. The smaller recent-match ages in R2 are descriptive observations, not proof that raising source rate improves freshness. More controller versions, longer stable-condition sessions, and dynamic measurements could reveal behaviors absent from this archive.

Internal identification is limited by missing receiver acceptance events, absent source timestamps in returns, and repeated acceleration values. The recent-match statistic is a compatible-history descriptor, and model agreement does not reveal which individual inputs were admitted. The residual disagreement, including failures around historical restoration, prevents claiming a fully reconstructed execution path. Complete-capture estimates deliberately include those transitions and should not be interpreted as steady-state admission probabilities. Timestamp sensitivity establishes stability over the specified offsets but cannot recover an unrecorded internal clock or scheduling trace.

The diagnostic couples checksum, marker, and order, and its windows are technically repeated rather than independent. A discriminating follow-up would randomize checksum classes and near-identical values, record a unique accepted-input identifier inside the receiver, and compare the same controller before and after a versioned correction. That experiment would separate checksum semantics from marker-dependent handling and test a repair on hardware. It would require a separately authorized procedure; none was carried out in this reanalysis.

The other decisive follow-up concerns control use. Logging the accepted sample, controller readout time, and synchronized physical response under a defined motion protocol would connect link behavior to control consequences. Without those observations, implications for gravity compensation, teleoperation, orientation estimation, protective behavior, or injury risk remain untested. No safety certification or motion-readiness claim follows from the present packet evidence.

Finally, the archive was collected for engineering diagnosis rather than a publication protocol, and some historical dependency versions and full acquisition metadata are missing. The package preserves these gaps. Network captures and controller context may require organizational authorization for public release, and proprietary binaries are excluded from the proposed supplement. Providing derived packet traces and independent comparison models supports auditability, but it cannot substitute for rights clearance or independent access to the physical controller.

6. Conclusions

This retrospective study verified robot IMU link integrity at distinct observation layers. Continuous, valid feedback and matches to earlier values coexisted with evidence of checksum-dependent reception during the measurement regime. Complete-capture comparisons, a repeated session, third-controller replay, a historical diagnostic, and offline checks supported that interpretation. End-of-session failures limited the candidate model beyond that regime. The practical contribution is a reproducible way to distinguish publication liveness from evidence of admitted and current measurements. The findings apply to the archived links and the specified comparison semantics; individual acceptance, true sensor age, corrected firmware behavior, and downstream motion effects require additional observations.

Supplementary Materials: Supplementary Information S1: analysis specification, diagnostic windows, timing sensitivity, complete-capture controls, transfer checks, environment and provenance; Supplementary Data S2: pseudonymized packet traces and offline analysis code; Supplementary Data S3: shifted ROS IMU derivatives and recording-semantics audit code, prepared locally for author review. Public release and repository identifiers remain subject to author authorization.

Author Contributions: To be completed by the authors using the CRediT roles, including acquisition of the historical engineering data, conceptualization, software, formal analysis, validation, writing, and supervision as applicable.

Funding: Funding sources, grant numbers, and support for publication charges require author confirmation.

Institutional Review Board Statement: Not applicable to the analyzed device telemetry and offline computations; no human-participant or animal experiment is reported.

Informed Consent Statement: Not applicable.

Data Availability Statement: Analysis code, parameter specifications, numerical results, and pseudonymized derivatives of the packet records, and shifted ROS IMU records have been prepared as supplementary materials. Original network captures and proprietary controller context are retained locally and are not publicly deposited. The authors must confirm the release permissions, access conditions, and final repository or supplement location before submission.

Acknowledgments: Technical contributors and organizational support require author confirmation.

Conflicts of Interest: Employment, commercial relationships, data ownership interests, and other potential conflicts require declarations from all authors.

References

1. Luckcuck, M.; Farrell, M.; Dennis, L.A.; Dixon, C.; Fisher, M. Formal Specification and Verification of Autonomous Robotic Systems: A Survey. *ACM Computing Surveys* **2019**, *52*, 100:1–100:41. https://doi.org/10.1145/3342355.
2. Araujo, H.; Mousavi, M.R.; Varshosaz, M. Testing, Validation, and Verification of Robotic and Autonomous Systems: A Systematic Review. *ACM Transactions on Software Engineering and Methodology* **2023**, *32*, 1–61. https://doi.org/10.1145/3542945.
3. Fisher, M.; Cardoso, R.C.; Collins, E.C.; Dadswell, C.; Dennis, L.A.; Dixon, C.; Farrell, M.; Ferrando, A.; Huang, X.; Jump, M.; et al. An Overview of Verification and Validation Challenges for Inspection Robots. *Robotics* **2021**, *10*, 67. https://doi.org/10.3390/robotics10020067.
4. Timperley, C.S.; van der Hoorn, G.; Santos, A.; Deshpande, H.; Wąsowski, A. ROBUST: 221 bugs in the Robot Operating System. *Empirical Software Engineering* **2024**, *29*, 57. https://doi.org/10.1007/s10664-024-10440-0.
5. Saltzer, J.H.; Reed, D.P.; Clark, D.D. End-to-end arguments in system design. *ACM Transactions on Computer Systems* **1984**, *2*, 277–288. https://doi.org/10.1145/357401.357402.

6. Zaki, O.; Dunnigan, M.; Robu, V.; Flynn, D. Reliability and Safety of Autonomous Systems Based on Semantic Modelling for Self-Certification. *Robotics* **2021**, *10*, 10. https://doi.org/10.3390/robotics10010010.
7. Ferrando, A.; Cardoso, R.C.; Fisher, M.; Ancona, D.; Franceschini, L.; Mascardi, V. ROSMonitoring: A Runtime Verification Framework for ROS. In Proceedings of the Towards Autonomous Robotic Systems. Springer, 2020, Vol. 12228, pp. 387–399. https://doi.org/10.1007/978-3-030-63486-5_40.
8. Ghaffari Saadat, M.; Ferrando, A.; Dennis, L.A.; Fisher, M. ROSMonitoring 2.0: Extending ROS Runtime Verification to Services and Ordered Topics. *Electronic Proceedings in Theoretical Computer Science* **2024**, *411*, 38–55. https://doi.org/10.4204/eptcs.411.3.
9. Caldas, R.; Piñera García, J.A.; Schiopu, M.; Pelliccione, P.; Rodrigues, G.; Berger, T. Runtime Verification and Field-Based Testing for ROS-Based Robotic Systems. *IEEE Transactions on Software Engineering* **2024**, *50*, 2544–2567. https://doi.org/10.1109/TSE.2024.3444697.
10. Kaul, S.; Yates, R.; Gruteser, M. Real-time status: How often should one update? In Proceedings of the IEEE INFOCOM, 2012, pp. 2731–2735. https://doi.org/10.1109/infcom.2012.6195689.
11. Yates, R.D.; Sun, Y.; Brown, D.R.; Kaul, S.K.; Modiano, E.; Ulukus, S. Age of Information: An Introduction and Survey. *IEEE Journal on Selected Areas in Communications* **2021**, *39*, 1183–1210. https://doi.org/10.1109/jsac.2021.3065072.
12. Burek, N. ROS QoS: Deadline, Liveliness, and Lifespan. Available online: https://design.ros2.org/articles/qos_deadline_liveliness_lifespan.html, 2019. Accessed on 19 October 2026.
13. Maxino, T.C.; Koopman, P.J. The Effectiveness of Checksums for Embedded Control Networks. *IEEE Transactions on Dependable and Secure Computing* **2009**, *6*, 59–72. https://doi.org/10.1109/tdsc.2007.70216.
14. Foote, T.; Purvis, M. REP-103: Standard Units of Measure and Coordinate Conventions. Available online: https://www.ros.org/reps/rep-0103.html, 2010. Accessed on 19 October 2026; verified against the ROS REP source repository.
15. MCAP Project. MCAP Format Specification. Available online: https://mcap.dev/spec. Accessed on 19 October 2026.
16. ROS 2 Common Interfaces Contributors. sensor_msgs/msg/Imu Message Definition. Available online: https://github.com/ros2/common_interfaces/blob/rolling/sensor_msgs/msg/Imu.msg. Rolling source definition; accessed on 19 October 2026; the recorded embedded schema was used for decoding.